



دانشگاه تربت حیدریه

دانشکده فنی مهندسی، گروه ICT
رشته فناوری اطلاعات و ارتباطات - مقطع کاردانی

گزارش نهایی پروژه
طراحی و پیاده سازی سیستم تشخیص چهره مبتنی بر
یادگیری عمیق
با استفاده از Python، OpenCV، InsightFace و
AuraFace
و کنترل LED توسط Arduino

نام دانشجو:

فاطمه یزدانی

نام استاد:

جناب آقای دشتبان

دانشگاه: فنی و مهندسی تربت حیدریه

تاریخ: 1405/05/15

این گزارش نسخه ارتقایافته پروژه تشخیص چهره است و منطق نرم افزاری آن بر پایه مدل AuraFace-v1 و استخراج بردار ویژگی چهره ((Face Embedding تدوین شده است.

چکیده

تشخیص چهره یکی از کاربردهای مهم بینایی ماشین و هوش مصنوعی است و در سامانه‌های کنترل دسترسی، حضور و غیاب، نظارت و تجهیزات هوشمند کاربرد دارد. در این پروژه یک سامانه تشخیص چهره با زبان برنامه‌نویسی Python طراحی و پیاده‌سازی شده است که علاوه بر پردازش تصویر و دریافت ویدئو از وبکم، نتیجه شناسایی را از طریق ارتباط سریال برای برد **Arduino** ارسال می‌کند. در نسخه جدید پروژه، هسته شناسایی از روش کلاسیک **LBPH** به یک رویکرد مبتنی بر یادگیری عمیق تغییر یافته و از **AuraFace-v1** و چارچوب **InsightFace** استفاده شده است. در این معماری، تصاویر مرجع فرد مجاز از پوشه مشخص‌شده خوانده می‌شوند. برای هر تصویر، چهره توسط موتور **InsightFace** شناسایی و یک بردار ویژگی نرمال‌شده یا **Face Embedding** استخراج می‌شود. در زمان اجرای دوربین نیز **embedding** چهره زنده استخراج شده و با **embedding**‌های مرجع مقایسه می‌شود. معیار مقایسه، **Cosine Similarity** است و بیشترین مقدار شباهت به عنوان بهترین تطابق در نظر گرفته می‌شود. مقدار شباهت با آستانه تعیین‌شده مقایسه می‌شود؛ در صورت عبور از آستانه، فرد مجاز شناخته شده و فرمان **M** برای **Arduino** ارسال می‌شود و در غیر این صورت فرمان **N** ارسال می‌گردد.

Arduino نقش کنترل‌کننده خروجی را بر عهده دارد و با دریافت **M**، **LED** سبز و با دریافت **N**، **LED** قرمز را فعال می‌کند. مدل **AuraFace** در اولین اجرا در صورت نبود نسخه محلی، از **Hugging Face** دریافت می‌شود و در مسیر محلی ذخیره می‌گردد. همچنین برنامه برای اجرای مدل از **GPU** دارای **CUDA** در صورت دسترسی پشتیبانی می‌کند و در غیر این صورت امکان اجرای **CPU** را دارد. هدف اصلی پروژه، ترکیب عملی هوش مصنوعی، پردازش تصویر، ارتباط سریال و کنترل یک سخت‌افزار ساده در قالب یک سامانه کنترل دسترسی آموزشی است.

واژگان کلیدی:

تشخیص چهره، یادگیری عمیق، Face Embedding، AuraFace، Cosine Similarity، Python، OpenCV، Arduino، کنترل دسترسی.

مقدمه

در سال‌های اخیر استفاده از سامانه‌های هوشمند مبتنی بر هوش مصنوعی و بینایی ماشین رشد چشمگیری داشته است. یکی از کاربردهای مهم این فناوری، تشخیص هویت افراد از روی ویژگی‌های چهره است. مزیت مهم این روش آن است که تشخیص می‌تواند بدون تماس فیزیکی و با استفاده از دوربین انجام شود. به همین دلیل، فناوری تشخیص چهره در حوزه‌هایی مانند کنترل ورود و خروج، حضور و غیاب، امنیت، نظارت و تجهیزات هوشمند مورد توجه قرار گرفته است.

یک سامانه تشخیص چهره معمولاً شامل دو وظیفه اصلی است: نخست، شناسایی محل چهره در تصویر و دوم، استخراج ویژگی و تعیین شباهت آن با نمونه‌های مرجع. در نسخه اولیه پروژه حاضر، این وظایف با روش‌های کلاسیک Haar Cascade و LBPH انجام می‌شدند. در نسخه ارتقایافته، ساختار به سمت یک مدل مبتنی بر یادگیری عمیق تغییر کرده است تا به جای توصیف بافتی ساده، ویژگی‌های عمیق و قابل مقایسه چهره در قالب embedding استخراج شود.

در این پروژه، نرم‌افزار Python مسئول تمام مراحل پردازش تصویر و تشخیص است. وبکم فریم‌های زنده را دریافت می‌کند و InsightFace چهره‌های موجود در هر فریم را تحلیل می‌نماید. سپس embedding چهره ورودی با embedding های مرجع مقایسه شده و نتیجه به دو حالت اصلی Match و Unknown تقسیم می‌شود. نتیجه نهایی به صورت یک کاراکتر از طریق پورت سریال به Arduino فرستاده می‌شود تا خروجی سخت‌افزاری متناسب با آن فعال شود. ساختار پروژه به گونه‌ای طراحی شده است که بخش هوشمند سامانه در رایانه اجرا شود و Arduino تنها فرمان نتیجه را دریافت و آن را به خروجی فیزیکی تبدیل کند. این جداسازی، طراحی پروژه را ساده، قابل فهم و قابل توسعه می‌کند و نشان می‌دهد چگونه می‌توان یک مدل هوش مصنوعی را با یک سیستم نهفته و کنترل سخت‌افزاری ترکیب کرد.

هدف پروژه

- طراحی و پیاده‌سازی یک سامانه تشخیص چهره با استفاده از Python و مدل AuraFace-v1.
- استفاده از InsightFace برای تشخیص چهره و استخراج Face Embedding.
- ایجاد مجموعه‌ای از embedding‌های مرجع از تصاویر فرد مجاز.
- مقایسه چهره ورودی با نمونه‌های مرجع با استفاده از Cosine Similarity.
- ارسال نتیجه تشخیص به Arduino از طریق ارتباط سریال.
- کنترل LED سبز و قرمز بر اساس نتیجه تشخیص.
- آشنایی عملی با مفاهیم پردازش تصویر، یادگیری عمیق، embedding و ارتباط نرم‌افزار و سخت‌افزار.

تجهیزات و نرم‌افزارهای مورد استفاده

1. سخت‌افزار

تجهیز	کاربرد
Arduino Uno	دریافت فرمان سریال و کنترل LEDها
Webcam	دریافت تصاویر زنده از چهره
LED سبز	نمایش شناسایی موفق
LED قرمز	نمایش فرد ناشناس
دو مقاومت 220 اهم	محدود کردن جریان LEDها
کابل USB	ارتباط رایانه با Arduino
بردبورد و سیم جامپر	پیاده‌سازی مدار آزمایشی

2. نرم‌افزار و کتابخانه‌ها

ابزار/کتابخانه	نقش در پروژه	نیاز اصلی
Python 3	اجرای برنامه اصلی	منطق پردازش و کنترل
OpenCV	دریافت فریم و نمایش ویدئو	تصویربرداری و رابط تصویری
InsightFace	تشخیص و تحلیل چهره	استخراج ویژگی چهره
AuraFace-v1	مدل یادگیری عمیق چهره	Face Embedding
NumPy	محاسبات برداری	Cosine Similarity

ابزار/کتابخانه	نقش در پروژه	نیاز اصلی
PySerial	ارتباط با Arduino	ارسال M/N
Hugging Face Hub و ONNX Runtime	دریافت و اجرای مدل	مدیریت و اجرای مدل

وابستگی‌های مورد نیاز نسخه جدید عبارت‌اند از:

insightface onnxruntime opencv-python numpy huggingface_hub pyserial

برای سیستم‌های دارای GPU سازگار با CUDA می‌توان از بسته gpu-onnxruntime به جای onnxruntime استفاده کرد.

آشنایی با فناوری‌های مورد استفاده

1. پردازش تصویر

پردازش تصویر مجموعه‌ای از روش‌ها برای دریافت، بهبود، تحلیل و استخراج اطلاعات از تصاویر دیجیتال است. در این پروژه پردازش تصویر در درجه اول برای دریافت فریم از دوربین، شناسایی چهره، رسم محدوده چهره و نمایش وضعیت شناسایی به کار می‌رود. در نسخه جدید، وظیفه تشخیص و استخراج ویژگی عمدتاً توسط InsightFace و مدل AuraFace انجام می‌شود؛ بنابراین بخش بیشتری از پردازش تخصصی چهره داخل pipeline مدل قرار گرفته است.

2. OpenCv

OpenCV یک کتابخانه متن‌باز و پرکاربرد در زمینه بینایی ماشین است. در این پروژه از OpenCV برای باز کردن دوربین، دریافت فریم، نمایش تصویر زنده، رسم مستطیل دور چهره و نوشتن متن نتیجه روی تصویر استفاده می‌شود. با وجود اینکه الگوریتم اصلی تشخیص چهره در نسخه جدید InsightFace است، OpenCV همچنان لایه مهم تصویربرداری و نمایش سامانه را تشکیل می‌دهد.

3. InsightFace

InsightFace یک چارچوب تخصصی برای تحلیل چهره است که در این پروژه به عنوان رابط اجرای مدل استفاده می‌شود. شیء FaceAnalysis مدل را آماده کرده و با تابع get، چهره‌های موجود

در تصویر را استخراج می‌کند. خروجی برای هر چهره شامل اطلاعات هندسی مانند محدوده چهره و همچنین ویژگی عددی آن یعنی embedding است. این ساختار باعث می‌شود تشخیص و استخراج ویژگی در یک pipeline یکپارچه انجام شود.

AuraFace-v1 .4

AuraFace-v1 مدل یادگیری عمیقی است که در این پروژه برای تولید ویژگی‌های چهره استفاده می‌شود. برنامه در اولین اجرا در صورت نبود مدل محلی، آن را از مخزن Hugging Face دریافت کرده و در مسیر models/auraface ذخیره می‌کند. به همین دلیل اجرای نخست ممکن است نسبت به اجراهای بعدی زمان بیشتری نیاز داشته باشد.

Face Embedding .5

Face Embedding یک نمایش عددی از ویژگی‌های چهره است. به جای مقایسه مستقیم دو تصویر، مدل یک چهره را به یک بردار عددی تبدیل می‌کند. این بردار تلاش می‌کند ویژگی‌های مهم چهره را در فضایی قرار دهد که چهره‌های مشابه به یکدیگر نزدیکتر باشند و چهره‌های متفاوت فاصله بیشتری داشته باشند. در این پروژه از embedding نرمال‌شده با نام normed_embedding استفاده می‌شود.

برای تصاویر مرجع، embedding هر تصویر استخراج و در حافظه نگهداری می‌شود. هنگام استفاده از دوربین، embedding چهره جدید نیز استخراج می‌شود و سپس با هر یک از embedding های مرجع مقایسه می‌گردد. در نتیجه، پروژه نیازمند آموزش مجدد مدل عمیق روی تصاویر شخصی نیست و تصاویر شخصی بیشتر نقش نمونه‌های مرجع برای مقایسه را دارند.

Cosine Similarity .6

برای اندازه‌گیری شباهت بین embedding جدید و embedding های مرجع، در برنامه از Cosine Similarity استفاده شده است. چون بردارها نرمال شده‌اند، ضرب داخلی دو بردار می‌تواند به عنوان معیار شباهت استفاده شود. تابع best_similarity برای هر embedding مرجع مقدار np.dot را محاسبه کرده و بیشترین مقدار را برمی‌گرداند. مقدار بیشتر نشان‌دهنده شباهت بیشتر است.

```

91 def best_similarity(embedding, reference_embeddings):
92     """Cosine similarity (dot product of normalized vectors) against the closest reference photo."""
93     return max(float(np.dot(embedding, ref)) for ref in reference_embeddings)
94

```

7. آستانه شباهت

پارامتر SIMILARITY_THRESHOLD در نسخه جدید نقش آستانه تصمیم‌گیری را دارد و مقدار اولیه آن 0.40 است. اگر similarity بزرگتر یا مساوی آستانه باشد، نتیجه Match است و اگر کمتر باشد، نتیجه Unknown خواهد بود. بنابراین برخلاف Confidence در LBPH، در این روش مقدار بالاتر به معنای شباهت بیشتر است و افزایش آستانه باعث سخت‌گیرانه‌تر شدن پذیرش چهره می‌شود.

معماری کلی سیستم

معماری سیستم از دو بخش اصلی نرم‌افزاری و سخت‌افزاری تشکیل شده است. بخش نرم‌افزاری روی رایانه اجرا شده و پردازش هوشمند را انجام می‌دهد. بخش سخت‌افزاری شامل Arduino و LEDهاست و نتیجه نرم‌افزار را به یک خروجی فیزیکی تبدیل می‌کند.

مرحله	ورودی	خروجی
تصاویر مرجع	فایل‌های JPG/PNG و غیره	Embedding های مرجع
دوربین	فریم زنده	چهره و embedding
مقایسه	Embedding جدید + مرجع	Similarity نهایی
تصمیم	Threshold + Similarity	Match یا Unknown
ارتباط سریال	M/N	فرمان برای Arduino

1. روند اجرای سیستم

1. شروع برنامه و بارگذاری مدل AuraFace-v1
2. دانلود مدل در اولین اجرا در صورت نبود نسخه محلی
3. خواندن تصاویر مرجع از پوشه images
4. تشخیص چهره و استخراج embedding از هر تصویر مرجع
5. برقراری ارتباط سریال با Arduino
6. فعال شدن دوربین و دریافت فریم‌های زنده
7. تشخیص چهره و استخراج embedding از هر فریم

- (8). محاسبه بیشترین Cosine Similarity با نمونه‌های مرجع
- (9). مقایسه similarity با آستانه 0.40
- (10). ارسال **M** برای Match یا **N** برای Unknown
- (11). کنترل LED توسط Arduino
- (12). ادامه فرایند تا فشردن **q** و آزادسازی منابع.

2. فلوچارت متنی

شروع برنامه



بارگذاری مدل AuraFace-v1



آماده سازی embedding های مرجع



برقراری ارتباط با Arduino



فعال سازی دوربین و دریافت فریم



تشخیص چهره با InsightFace



استخراج Face Embedding



محاسبه بیشترین Cosine Similarity



مقایسه با آستانه 0.40



در صورت تطابق: ارسال M → LED سبز

در صورت عدم تطابق: ارسال N → LED قرمز



ادامه تا فشردن q و آزادسازی منابع

تحلیل و توضیح کد Python

1. توضیحات ابتدایی برنامه

در ابتدای فایل، توضیحاتی درباره هدف برنامه، روش استفاده از مدل، نحوه ارتباط با Arduino و فرمان خروجی درج شده است. برنامه اعلام می کند که از تمام تصاویر مرجع موجود در IMAGES_DIR برای استخراج یک embedding از هر تصویر استفاده می شود، دوربین فعال شده و embedding چهره های زنده با embedding های مرجع از طریق Cosine Similarity مقایسه می گردد. همچنین فرمان M یا N برای Arduino ارسال می شود.

2. کتابخانه‌ها

os برای مدیریت پوشه‌ها، time برای تأخیر و زمان‌بندی، cv2 برای تصویربرداری و نمایش، numpy برای محاسبات برداری، serial برای ارتباط با Arduino، snapshot_download برای دریافت مدل از Hugging Face و FaceAnalysis برای راه‌اندازی pipeline تشخیص چهره استفاده می‌شوند.

```
22 import os
23 import time
24
25 import cv2
26 import numpy as np
27 import serial
28 from huggingface_hub import snapshot_download
29 from insightface.app import FaceAnalysis
30
```

3. تنظیمات اصلی

قرار دادن تنظیمات در ابتدای برنامه باعث می‌شود کاربر بدون ورود به توابع اصلی بتواند پورت Arduino، سرعت سریال، مسیر تصاویر، دوربین، آستانه تشخیص، فاصله زمانی ارسال و مسیر مدل را تغییر دهد.

```
1 SERIAL_PORT = "COM3" # Arduino port, e.g. "COM3" (Windows) or "/dev/ttyUSB0" (Linux/Mac)
2 BAUD_RATE = 9600 # must match Serial.begin(...) in the Arduino code
3 IMAGES_DIR = "images" # folder with reference photos of the authorized person
4 CAMERA_INDEX = 0 # webcam index (0 = default camera)
5 SIMILARITY_THRESHOLD = 0.40 # higher = stricter match (try values between 0.35 and 0.60)
6 SEND_INTERVAL = 1.5 # min seconds between repeated serial sends of the same result
7 MODEL_DIR = "models/auraface" # local cache folder for the downloaded model
```

4. تابع load_face_model

این تابع مسئول دانلود یکباره و راه‌اندازی مدل است. ابتدا وجود مسیر مدل بررسی می‌شود. اگر پوشه مدل آماده نباشد، snapshot_download مدل AuraFace-v1 را دریافت می‌کند. سپس FaceAnalysis با نام auraface ساخته می‌شود. برنامه دو provider دارد: CUDAEExecutionProvider و CPUExecutionProvider. بنابراین اگر محیط مناسب GPU وجود داشته باشد، اجرای محاسبات می‌تواند

از آن استفاده کند و در غیر این صورت CPU به عنوان مسیر پشتیبان در دسترس است.

```
42 def load_face_model():
43     """Download (once) and initialize the AuraFace-v1 face analysis model."""
44     if not os.path.exists(os.path.join(MODEL_DIR, "models")):
45         print("Downloading AuraFace-v1 model from Hugging Face (first run only)...")
46         snapshot_download(repo_id="fal/AuraFace-v1", local_dir=MODEL_DIR)
47         print("Download finished.")
48
49     app = FaceAnalysis(
50         name="auraface",
51         root=".",
52         providers=["CUDAExecutionProvider", "CPUExecutionProvider"], # falls back to CPU automatically
53     )
54     app.prepare(ctx_id=0, det_size=(640, 640))
55     return app
```

5. تابع load_reference_embeddings

این تابع جایگاه مهمی در نسخه جدید دارد. برنامه همه فایل‌های تصویری مجاز را از پوشه images پیدا می‌کند. اگر فایل قابل خواندن نباشد یا چهره‌ای در آن یافت نشود، فایل رد می‌شود. اگر چند چهره در تصویر وجود داشته باشد، بزرگترین چهره انتخاب می‌شود و embedding نرمال‌شده آن داخل آرایه مرجع قرار می‌گیرد.

```
58 def load_reference_embeddings(face_app, images_dir):
59     """Extract one face embedding per reference photo found in images_dir."""
60     valid_ext = (".jpg", ".jpeg", ".png", ".bmp")
61
```

6. تابع best_similarity

این تابع شباهت چهره ورودی با همه نمونه‌های مرجع را بررسی می‌کند. چون یک فرد می‌تواند چند تصویر مرجع داشته باشد، برنامه تنها یک مقایسه انجام نمی‌دهد؛ بلکه similarity تمام نمونه‌ها را محاسبه و بیشترین آن را انتخاب می‌کند. این روش کمک می‌کند اگر یکی از تصاویر مرجع از نظر نور یا زاویه مناسب نباشد، نمونه دیگری بتواند شباهت بالاتری ایجاد کند.

```
91 def best_similarity(embedding, reference_embeddings):
92     """Cosine similarity (dot product of normalized vectors) against the closest reference photo."""
93     return max(float(np.dot(embedding, ref)) for ref in reference_embeddings)
94
```

7. تابع connect_to_arduino

تابع `connect_to_arduino` ارتباط سریال را ایجاد می‌کند. در صورت موفقیت، شیء `Serial` بازگردانده می‌شود و برنامه دو ثانیه صبر می‌کند تا برد `Arduino` فرصت راه‌اندازی و `Reset` اولیه را داشته باشد. اگر خطایی رخ دهد، `None` برگردانده می‌شود تا بخش تشخیص چهره همچنان قابل اجرا باشد.

```
96 def connect_to_arduino(port, baud):
97     """Try connecting to Arduino; return None on failure (program still runs without serial)."""
98     try:
99         ser = serial.Serial(port, baud, timeout=1)
100         time.sleep(2) # give the Arduino time to reset after the port opens
101         print(f"Connected to Arduino on {port}.")
102         return ser
103     except Exception as e:
104         print(f"Warning: could not connect to Arduino ({e}). Continuing without serial.")
105     return None
```

8. تابع main

تابع `main` نقطه شروع اجرای منطق اصلی است. ابتدا مدل بارگذاری می‌شود، سپس `embedding` های مرجع استخراج می‌شوند، ارتباط با `Arduino` برقرار می‌شود و دوربین باز می‌گردد. پس از آن حلقه اصلی برنامه تا زمان خروج کاربر اجرا خواهد شد.

9. دریافت فریم و تشخیص چهره

در هر تکرار یک فریم از وبکم دریافت می‌شود. اگر دریافت ناموفق باشد، برنامه آن تکرار را کنار می‌گذارد. در صورت موفقیت، `frame` به `InsightFace` داده می‌شود تا چهره‌های موجود در آن شناسایی شوند.

```
127 ret, frame = cam.read()
128 if not ret:
129     continue
130
131 faces = face_app.get(frame)
132 current_result = None
133
```

. محاسبه شباهت و تصمیم‌گیری

این بخش هسته تصمیم‌گیری است. برای هر چهره مقدار similarity محاسبه می‌شود. اگر مقدار آن از آستانه تعیین‌شده بیشتر یا مساوی باشد، وضعیت match فعال می‌شود و کادر سبز و متن Match نمایش داده می‌شود؛ در غیر این صورت وضعیت mismatch و متن Unknown نمایش داده می‌شود.

```
134     for face in faces:
135         sim = best_similarity(face.normed_embedding, reference_embeddings)
136         x1, y1, x2, y2 = face.bbox.astype(int)
137
138         if sim >= SIMILARITY_THRESHOLD:
139             current_result, color, text = "match", (0, 255, 0), f"Match ({sim:.2f})"
140         else:
141             current_result, color, text = "mismatch", (0, 0, 255), f"Unknown ({sim:.2f})"
```

11. رسم کادر و متن روی تصویر

محدوده هر چهره از bbox مدل گرفته می‌شود و به مختصات صحیح تبدیل می‌گردد. سپس مستطیل دور چهره رسم و نتیجه همراه با مقدار similarity روی تصویر نمایش داده می‌شود.

```
143     cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
144     cv2.putText(frame, text, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)
```

12. ارسال فرمان به Arduino

برای جلوگیری از ارسال مداوم یک نتیجه ثابت، دو متغیر last_sent_time و last_sent_state نگهداری می‌شوند. اگر نتیجه تغییر کند، فرمان جدید بلافاصله ارسال می‌شود. همچنین اگر نتیجه ثابت بماند ولی از ارسال قبلی بیش از SEND_INTERVAL ثانیه گذشته باشد، فرمان دوباره ارسال می‌شود. این منطق باعث کاهش ارسال‌های تکراری روی پورت سریال می‌شود.

```
149     if current_result != last_sent_state or (now - last_sent_time) > SEND_INTERVAL:
150         cmd = "M" if current_result == "match" else "N"
151         try:
152             ser.write(cmd.encode())
```

13. پایان امن برنامه

در بلوک finally، دوربین آزاد می‌شود، پنجره‌های OpenCV بسته می‌شوند و در صورت وجود اتصال سریال، پورت Arduino نیز بسته

می‌شود. استفاده از `finally` باعث می‌شود حتی در صورت رخ دادن خطا یا خروج از حلقه، منابع اصلی تا حد امکان آزاد شوند.

```
160         finally:
161             cam.release()
162             cv2.destroyAllWindows()
163             if ser is not None:
164                 ser.close()
165
```

14. شرط اجرای مستقیم

در انتهای فایل شرط `_name_` بررسی می‌شود. در صورتی که فایل مستقیماً اجرا شده باشد، تابع `main` فراخوانی خواهد شد. این ساختار یکی از الگوهای استاندارد `Python` برای مشخص کردن نقطه شروع اجرای برنامه است.

بررسی برنامه Arduino

در این پروژه `Arduino` مسئول تشخیص چهره نیست. تمام پردازش‌های هوشمند در `Python` انجام می‌شوند و `Arduino` تنها نتیجه را از طریق ارتباط سریال دریافت کرده و خروجی فیزیکی را کنترل می‌کند. این جداسازی وظایف باعث می‌شود بار محاسباتی سنگین روی رایانه باقی بماند و برد `Arduino` با منطق ساده و قابل اطمینان، `LED`ها را کنترل کند.

1. تعریف پایه‌های خروجی

در این ساختار `LED` قرمز به پایه دیجیتال 10 و `LED` سبز به پایه دیجیتال 11 متصل می‌شوند. استفاده از متغیر ثابت باعث می‌شود در صورت تغییر سیم‌بندی بتوان تنها شماره پایه‌ها را تغییر داد. در این ساختار `LED` قرمز به پایه دیجیتال 10 و `LED` سبز به پایه دیجیتال 11 متصل می‌شوند. استفاده از متغیر ثابت باعث می‌شود در صورت تغییر سیم‌بندی بتوان تنها شماره پایه‌ها را تغییر داد.

```
const int RED_PIN = 10;
const int GREEN_PIN = 11;
```

2. تابع setup

```
void setup() {  
    pinMode(RED_PIN, OUTPUT);  
    pinMode(GREEN_PIN, OUTPUT);  
  
    Serial.begin(9600);  
  
    digitalWrite(RED_PIN, LOW);  
    digitalWrite(GREEN_PIN, LOW);  
}
```

تابع setup تنها یک بار هنگام روشن شدن یا Reset برد اجرا می‌شود. در آن پایه‌های LED خروجی می‌شوند، ارتباط سریال با نرخ 9600 بیت بر ثانیه آغاز می‌شود و هر دو LED در وضعیت خاموش قرار می‌گیرند.

3. تابع loop

```
void loop() {  
    if (Serial.available() > 0) {  
        char cmd = Serial.read();  
  
        if (cmd == 'M') {  
            digitalWrite(GREEN_PIN, HIGH);  
            digitalWrite(RED_PIN, LOW);  
        } else if (cmd == 'N') {  
            digitalWrite(RED_PIN, HIGH);  
            digitalWrite(GREEN_PIN, LOW);  
        }  
    }  
}
```

تابع loop به صورت پیوسته اجرا شده و در انتظار داده سریال می‌ماند. دریافت M به معنای شناسایی موفق است و LED سبز را روشن می‌کند. دریافت N به معنای عدم تطابق است و LED قرمز را روشن می‌کند.

ارتباط سریال Python و Arduino

ارتباط میان Python و Arduino از طریق USB و پورت سریال انجام می‌شود. نرخ انتقال داده در هر دو طرف 9600 بیت بر ثانیه تنظیم شده است. نرم‌افزار Python پس از تصمیم‌گیری تنها یک کاراکتر برای Arduino ارسال می‌کند. این طراحی ساده است، حجم داده کمی دارد و برای یک پروژه آموزشی کنترل دسترسی مناسب است.

کاراکتر	معنی	خروجی Arduino
M	شناسایی موفق	LED سبز روشن، قرمز خاموش
N	عدم تطابق	LED قرمز روشن، سبز خاموش

1. مدیریت خطای ارتباطی

برنامه Python تلاش می‌کند اتصال سریال را برقرار کند. اگر پورت اشتباه باشد یا Arduino در دسترس نباشد، برنامه با پیام هشدار ادامه می‌دهد و بخش تشخیص چهره همچنان می‌تواند اجرا شود. در زمان ارسال نیز خطاهای Serial Write در بلوک try/except کنترل می‌شوند. این رویکرد باعث می‌شود خطای سخت‌افزار لزوماً موجب توقف کامل پردازش تصویر نشود.

2. نقش SEND_INTERVAL

پارامتر SEND_INTERVAL حداقل فاصله زمانی میان ارسال‌های تکراری یک نتیجه ثابت را مشخص می‌کند. مقدار پیش‌فرض 1.5 ثانیه است. اگر وضعیت تغییر کرده باشد، فرمان جدید بدون انتظار برای پایان این بازه ارسال می‌شود؛ اما اگر وضعیت ثابت بماند، ارسال‌های مجدد با فاصله زمانی مشخص محدود می‌شوند.

شرایط اجرا و نکات عملی

- پوشه images باید وجود داشته باشد و حداقل یک تصویر قابل خواندن از فرد مجاز در آن قرار گیرد.
- تصاویر مرجع بهتر است چهره واضح، نور مناسب و پوشش محدود داشته باشند.
- اجرای اول ممکن است به دلیل دانلود مدل AuraFace زمان بیشتری نیاز داشته باشد.

- پورت COM8 باید مطابق پورت واقعی Arduino تنظیم شود.
- سرعت سریال Python و Arduino باید یکسان باشد.
- برای دسترسی به GPU، محیط CUDA و provider مناسب باید به درستی نصب و شناسایی شود.
- مقدار SIMILARITY_THRESHOLD باید با آزمایش روی محیط واقعی تنظیم شود و از ادعای یک دقت ثابت بدون اندازه‌گیری باید خودداری شود.

نتیجه اجرای پروژه

در اجرای صحیح پروژه، برنامه ابتدا مدل تشخیص چهره را بارگذاری می‌کند. در اولین اجرا، در صورت نبود نسخه محلی، مدل AuraFace از Hugging Face دریافت می‌شود. سپس تصاویر موجود در پوشه images خوانده شده و از چهره موجود در هر تصویر، embedding نرمال‌شده استخراج می‌شود. این embedding ها به عنوان نمونه‌های مرجع در حافظه نگهداری می‌شوند.

پس از آماده‌سازی داده‌های مرجع، ارتباط سریال با Arduino برقرار شده و دوربین فعال می‌شود. در هر فریم، چهره‌های موجود شناسایی می‌شوند و برای هر چهره embedding استخراج می‌شود. این embedding با تمام نمونه‌های مرجع مقایسه و بیشترین similarity انتخاب می‌شود. مقدار به دست آمده روی تصویر نمایش داده می‌شود و بر اساس آستانه تصمیم‌گیری، چهره به عنوان Match یا Unknown دسته‌بندی می‌شود.

در صورت Match، فرمان M ارسال شده و Arduino LED سبز را روشن می‌کند. در صورت Unknown، فرمان N ارسال شده و LED قرمز روشن می‌شود. این روند تا زمانی که کاربر کلید q را فشار دهد تکرار می‌شود. سپس منابع شامل دوربین، پنجره‌های OpenCV و اتصال سریال آزاد می‌شوند.

1. مشاهده‌های قابل انتظار

وضعیت	نمایش روی تصویر	فرمان
فرد مجاز	similarity + Match بالا	M
فرد ناشناس	similarity + Unknown پایین‌تر از آستانه	N
عدم تشخیص چهره	بدون نتیجه تشخیص	بدون فرمان جدید
قطع ارتباط Arduino	تشخیص ادامه دارد	ارسال انجام نمی‌شود

2. مزایای نسخه ارتقا یافته

- استفاده از مدل مبتنی بر یادگیری عمیق به جای تشخیص هویت با LBPH.
- استفاده از Face Embedding برای نمایش عددی ویژگی‌های چهره.
- مقایسه چند تصویر مرجع و انتخاب بهترین similarity.
- امکان استفاده از GPU دارای CUDA در محیط مناسب و پشتیبانی از CPU به عنوان مسیر پشتیبان.
- کاهش پیچیدگی سمت Arduino؛ زیرا برد فقط فرمان نتیجه را دریافت می‌کند.
- ساختار ماژولار شامل توابع جداگانه برای مدل، embedding‌های مرجع، ارتباط سریال و حلقه اصلی.
- قابلیت توسعه برای کنترل دسترسی، حضور و غیاب و پروژه‌های هوشمند مشابه.

نتیجه‌گیری

در این پروژه یک سامانه تشخیص چهره و کنترل خروجی سخت‌افزاری با استفاده از Python، OpenCV، InsightFace، مدل AuraFace-v1 و Arduino طراحی و پیاده‌سازی شد. مهم‌ترین ارتقای انجام‌شده، انتقال هسته شناسایی از روش کلاسیک LBPH به یک روش مبتنی بر یادگیری عمیق و Face Embedding است. در این ساختار، تصاویر مرجع به بردارهای ویژگی نرمال‌شده تبدیل می‌شوند و چهره ورودی نیز به همین روش نمایش عددی پیدا می‌کند.

سپس با استفاده از Cosine Similarity نزدیکترین نمونه مرجع تعیین و بر اساس Threshold درباره Match یا Unknown تصمیم‌گیری می‌شود. نتیجه تصمیم‌گیری از طریق ارتباط سریال برای Arduino ارسال می‌شود. فرمان M نشان‌دهنده شناسایی موفق و فرمان N نشان‌دهنده عدم تطابق است. Arduino نیز بر اساس فرمان دریافت‌شده، LED سبز یا قرمز را فعال می‌کند. این تقسیم وظایف باعث می‌شود پردازش سنگین هوش مصنوعی روی رایانه انجام شود و Arduino نقش کنترل‌کننده ساده و قابل اتکا را داشته باشد.

ساختار نهایی پروژه از نظر آموزشی نمونه‌ای مناسب برای ترکیب بینایی ماشین، یادگیری عمیق، پردازش برداری، ارتباط سریال و کنترل سخت‌افزار است. همچنین با افزودن پایگاه داده، چند کاربر، رابط کاربری، قفل الکترونیکی و الگوریتم‌های امنیتی تکمیلی می‌توان آن را به یک سامانه کنترل دسترسی کامل‌تر تبدیل کرد.

منابع و مراجع

- کد پروژه Python ارائه‌شده برای نسخه ارتقایافته سیستم تشخیص چهره با AuraFace و InsightFace
- مستندات و بسته نرم‌افزاری InsightFace مورد استفاده در اجرای FaceAnalysis
- مدل AuraFace-v1 ارائه‌شده از طریق مخزن Hugging Face.
- مستندات OpenCV برای دریافت فریم، نمایش تصویر و پردازش ویدئویی.
- مستندات Python و کتابخانه NumPy برای محاسبات عددی و برداری.
- مستندات PySerial برای برقراری ارتباط سریال بین Python و Arduino
- مستندات Arduino IDE و Arduino Uno برای برنامه‌نویسی و کنترل خروجی‌های دیجیتال.

پیوست: ساختار نهایی کد Python

ساختار کلی فایل نهایی برنامه به صورت زیر است. این بخش برای مرور سریع اجزای پروژه ارائه شده است و نشان می‌دهد هر قسمت چه مسئولیتی دارد.

بخش کد	وظیفه
تنظیمات	تعریف پورت، دوربین، آستانه، مسیر تصاویر و مدل
load_face_model	دانلود یکباره و راه‌اندازی AuraFace
load_reference_embeddings	استخراج embedding از تصاویر مرجع
best_similarity	محاسبه و انتخاب بیشترین similarity
connect_to_arduino	برقراری ارتباط سریال
main	اجرای چرخه اصلی تشخیص و ارسال فرمان
finally	آزادسازی دوربین، پنجره‌ها و پورت سریال

پایان